

D3.js per trasformare la struttura logica e gerarchica dell'albero in coordinate cartesiane bidimensionali, evitando sovrapposizioni tra i nodi.

Metodi

- + `calculateLayout(treeMap: Map<String, Node>, rootId: String): LayoutResult` — riceve la mappa logica dei nodi e restituisce il `LayoutResult` contenente nodi e archi con le coordinate X e Y calcolate.
- - `extractNodesArray(treeMap: Map): List<Node>` — metodo di utilità interno per la conversione della mappa in array.

5.9.2.3) TreeCanvas

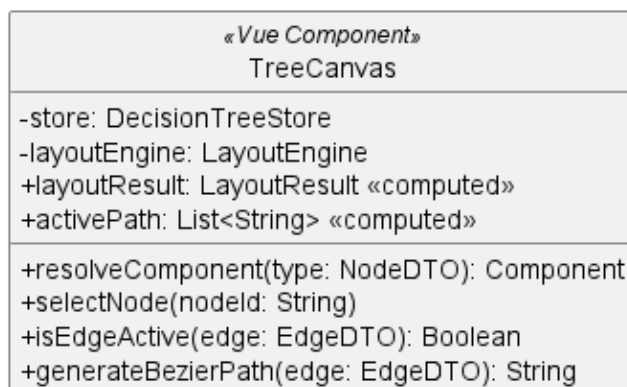


Figura 178: TreeCanvas

Descrizione

TreeCanvas è il componente Vue.js principale per la visualizzazione dell'albero. Interroga lo Store, passa i dati al Layout Engine per il calcolo geometrico e si occupa di orchestrare il rendering dinamico di linee e nodi a schermo.

Proprietà Compute e Stato

- - `store: DecisionTreeStore` — riferimento allo stato globale.
- - `layoutEngine: LayoutEngine` — riferimento al motore matematico.
- + `layoutResult: LayoutResult` — (computed) si aggiorna automaticamente se l'albero cambia, innescando un nuovo ricalcolo geometrico.
- + `activePath: List<String>` — (computed) recupera l'elenco degli ID dei nodi correntemente attivi in base alle risposte dell'utente.

Metodi

- + `resolveComponent(type: NodeDTO): Component` — determina dinamicamente quale sotto-componente Vue renderizzare (*UiDecisionNode* o *UiLeafNode*) in base al tipo del nodo.
- + `selectNode(nodeId: String)` — gestisce il click su un nodo della mappa, notificando lo Store.

- + `isEdgeActive(edge: EdgeDTO): Boolean` — verifica se la linea di collegamento appartiene al percorso attivo per applicare stili di evidenziazione.
- + `generateBezierPath(edge: EdgeDTO): String` — calcola la stringa del tracciato SVG per disegnare una curva morbida tra due nodi.

5.9.2.4) UiDecisionNode e UiLeafNode

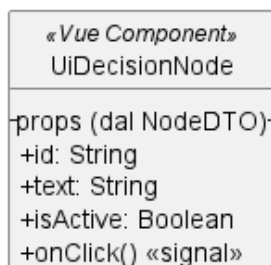


Figura 179: UiDecisionNode e UiLeafNode

Descrizione

UiDecisionNode e *UiLeafNode* sono i micro-componenti visivi montati dinamicamente dal *TreeCanvas*. Si occupano unicamente di stampare a schermo l'interfaccia di un singolo nodo, applicando stili CSS differenti a seconda che il nodo sia attivo o meno, e intercettando le interazioni dell'utente.

Proprietà (Props passate dal padre)

- + `id: String` — identificativo del nodo.
- + `text: String` — testo da mostrare (domanda o esito).
- + `isActive: Boolean` — flag iniettato dal *TreeCanvas* per indicare se il nodo è parte del percorso selezionato.
- + `resultState: String` — (Solo in *UiLeafNode*) indica l'esito finale della valutazione per colorare opportunamente il nodo (es. verde/rosso).

Eventi (Signals)

- + `onClick()` — (Solo in *UiDecisionNode*) emesso quando l'utente fa click sulla forma geometrica del nodo, catturato dal *TreeCanvas*.

5.9.3) Core di Valutazione (Domain Logic Client-side)

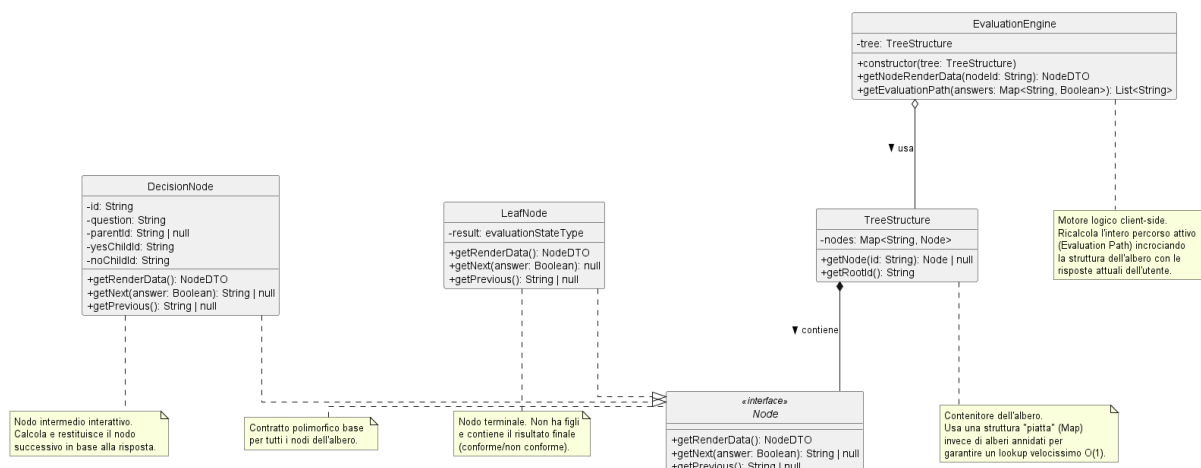


Figura 180: Diagramma delle classi del Motore di Valutazione

Descrizione

Il «Cervello» del modulo frontend risiede in una gerarchia di classi che implementano la logica di navigazione dell'albero decisionale direttamente nel browser. Questo approccio permette di ricalcolare il percorso di valutazione istantaneamente a ogni clic dell'utente, senza richiedere l'intervento del server. Il sistema utilizza il polimorfismo per distinguere il comportamento tra nodi di domanda e nodi di esito.

5.9.3.1) Interfaccia Node

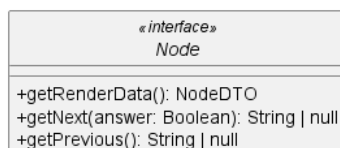


Figura 181: Interfaccia Node

Descrizione

Node è l'interfaccia (o contratto) base che definisce il comportamento comune a tutti i tipi di nodi presenti nell'albero. Garantisce che ogni nodo, indipendentemente dalla sua natura, possa fornire i propri dati di rendering e gestire la navigazione.

Metodi

- + `getRenderData(): NodeDTO` — restituisce i dati necessari al componente visivo per mostrare il nodo.
- + `getNext(answer: Boolean): String | null` — calcola l'ID del nodo successivo in base alla risposta ricevuta.
- + `getPrevious(): String | null` — restituisce l'ID del nodo genitore per permettere la navigazione a ritroso.

5.9.3.2) DecisionNode

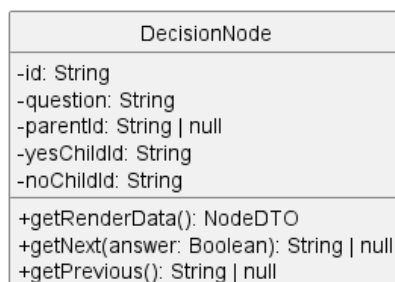


Figura 182: DecisionNode

Descrizione

DecisionNode è la classe che implementa un nodo intermedio (domanda). Contiene la logica decisionale binaria (Sì/No) per determinare quale ramo dell'albero debba essere percorso.

Attributi

- - id, - question: String — identificativo e testo della domanda.
- - parentId: String | null — riferimento al nodo precedente.
- - yesChildId, - noChildId: String — riferimenti ai due possibili nodi successivi.

Metodi

- + getNext(answer: Boolean) — implementazione concreta che restituisce yesChildId se la risposta è true, altrimenti noChildId.

5.9.3.3) LeafNode

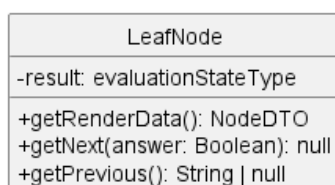


Figura 183: LeafNode

Descrizione

LeafNode rappresenta il punto terminale di un ramo dell'albero (foglia). Non permette ulteriori avanzamenti e contiene lo stato finale della valutazione per quel determinato percorso.

Attributi

- - result: evaluationStateType — definisce l'esito finale (es. «Conforme», «Non Conforme»).

Metodi

- + getNext(answer: Boolean) — restituisce sempre null, segnando la fine del percorso.

5.9.3.4) TreeStructure

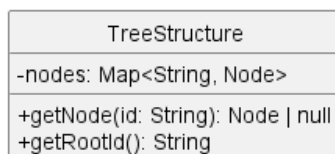


Figura 184: TreeStructure

Descrizione

TreeStructure funge da contenitore organizzato per l'intero albero decisionale. Al fine di massimizzare le prestazioni di ricerca lato client, utilizza una rappresentazione «piatta» basata su una mappa anziché una struttura nidificata.

Attributi

- - nodes: Map<String, Node> — mappa che indicizza tutti i nodi tramite il loro ID univoco per un accesso immediato ($O(1)$).

Metodi

- + getNode(id: String): Node | null — recupera l'oggetto nodo corrispondente all'ID fornito.
- + getRootId(): String — restituisce l'ID del punto di partenza della valutazione.

5.9.3.5) EvaluationEngine

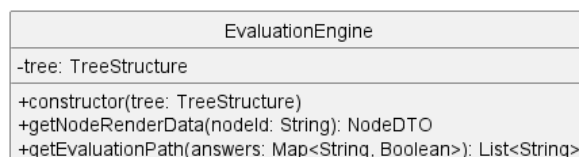


Figura 185: EvaluationEngine

Descrizione

EvaluationEngine è il motore logico principale che orchestra la valutazione. È il componente che traduce la massa di risposte dell'utente in un percorso visivo coerente e navigabile.

Attributi

- - tree: TreeStructure — la struttura dell'albero su cui operare.

Metodi

- + `getEvaluationPath(answers: Map<String, Boolean>): List<String>` — incrocia la mappa delle risposte fornite dall'utente con la struttura dell'albero per generare la lista ordinata di ID che compongono l'attuale «Percorso Attivo» (Active Path).
- + `getNodeRenderData(nodeId: String): NodeDTO` — metodo di utilità per ottenere i dati grafici di un nodo specifico tramite il motore.

5.9.4) DecisionTreeStore

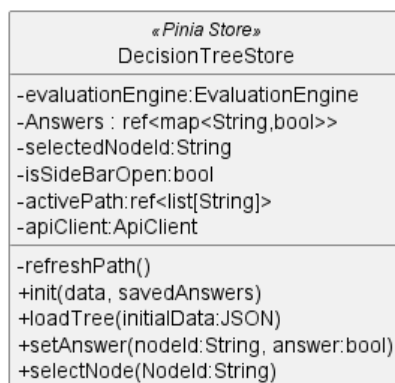


Figura 186: Dettaglio del DecisionTreeStore (Pinia)

Descrizione

DecisionTreeStore rappresenta il cuore reattivo dell'applicazione. È lo Store globale che permette la comunicazione disaccoppiata tra i vari componenti Vue (Canvas e Sidebar). Invece di passare dati tramite «props» e «events» complessi, i componenti leggono e scrivono direttamente in questo store, che si occupa di mantenere la coerenza dei dati.

Attributi (State)

- - `evaluationEngine: EvaluationEngine` — istanza del motore logico utilizzata per i calcoli di navigazione.
- - `answers: ref<Map<String, Boolean>>` — stato reattivo delle risposte fornite dall'utente durante la sessione corrente.
- - `selectedNodeId: String` — identifica il nodo su cui l'utente sta interagendo o che ha cliccato nel Canvas.
- - `activePath: ref<List<String>>` — lista reattiva degli ID dei nodi che compongono il percorso corrente, aggiornata automaticamente al variare delle risposte.
- - `apiClient: ApiClient` — riferimento al client per le chiamate verso le API del backend.

Metodi (Actions)

- + `init(data, savedAnswers)` — inizializza lo store caricando la struttura dell'albero e le eventuali risposte già salvate nel database.
- + `setAnswer(nodeId, answer)` — aggiorna una risposta nello stato locale, invoca il ricalcolo del percorso e avvia la persistenza asincrona tramite l'API.

- + `selectNode(nodeId)` — aggiorna il nodo selezionato, permettendo alla Sidebar di mostrare le informazioni contestuali corrette.
- - `refreshPath()` — metodo privato invocato internamente per aggiornare l'attributo `activePath` interpellando l'*EvaluationEngine*.

5.9.5) EvaluationApiClient

EvaluationApiClient
-baseUrl: String
+saveAnswer(answer: bool, device_id: String, asset_id: String, requirement_id: String, node_id: String): Promise<JSON> +fetchRequirementEvaluationState(device_id: String, asset_id: String, requirement_id: String): Promise<JSON>

Figura 187: Dettaglio dell'EvaluationApiClient

Descrizione

EvaluationApiClient è l'Outbound Adapter del frontend. Questa classe incapsula tutta la logica di comunicazione HTTP, isolando il resto dell'applicazione (Store e Componenti) dai dettagli implementativi delle chiamate API. Utilizza il pattern delle *Promise* per gestire l'asincronia tipica delle richieste di rete.

Attributi

- - `baseUrl: String` — l'indirizzo radice del server API.

Metodi

- + `saveAnswer(answer, device_id, asset_id, requirement_id, node_id): Promise<JSON>` — invia una richiesta asincrona al server per salvare la risposta fornita a un determinato nodo dell'albero. Riceve tutti i parametri di contesto necessari (dispositivo, asset e requisito) per garantire la corretta associazione dei dati nel database.
- + `fetchRequirementEvaluationState(device_id, asset_id, requirement_id): Promise<JSON>` — recupera dal backend lo stato attuale della valutazione per un determinato requisito. Questo metodo è fondamentale durante la fase di inizializzazione per ripristinare il «Percorso Attivo» basandosi sui dati salvati in precedenza.

6) Tracciamento

7) Gestione degli Errori

L'applicazione adotta una strategia di gestione degli errori stratificata. Le eccezioni non attraversano liberamente i confini architetturali, vengono invece intercettate, contestualizzate e tradotte man mano che si propagano dai livelli più profondi verso l'esterno. Per garantire chiarezza d'intenti e manutenibilità del codice, l'implementazione si basa su una netta distinzione semantica definita dai suffissi delle classi:

- **Suffisso **Error* (Livello di Dominio e Infrastruttura):** Viene utilizzato all'interno del Domain Layer (ereditando da una classe base `DomainError`) per segnalare la violazione di regole di business, oppure negli Outbound Adapter per segnalare problemi di interazione con risorse esterne o fallimenti infrastrutturali.
- **Suffisso **Failure* (Livello Applicativo / Casi d'Uso):** Identifica il fallimento di un intero flusso di lavoro orchestrato da un **Application Service**. Queste eccezioni descrivono l'impossibilità di portare a termine un'operazione dal punto di vista dell'utente o del sistema chiamante.

7.0.1) Il Flusso di Traduzione degli Errori (Exception Mapping)

L'intento di questa gestione degli errori è di evitare che dettagli tecnici (come l'uso specifico di MongoDB) inquinino i livelli superiori. Il meccanismo di propagazione si articola in tre fasi:

1. **Adattatori di Output (Infrastruttura):** Sollevano eccezioni custom di tipo `Error` per segnalare l'impossibilità di completare un'operazione di propria competenza. Questo avviene sia intercettando e traducendo errori generati da librerie di terze parti (es. eccezioni dei driver del database), sia generandole direttamente in base alla logica interna dell'adapter (es. risorse non trovate in un sistema di cache in memoria).
2. **Application Services (Casi d'Uso):** Durante l'orchestrazione, intercettano le eccezioni `Error` provenienti dal dominio o dai repository e le sollevano nuovamente (wrapping) traducendole in eccezioni di tipo `Failure`, aggiungendo il contesto specifico dell'operazione in corso.
3. **Adattatori di Input (Controller):** Comunicano esclusivamente con i Service, ricevendo solo eccezioni `Failure`. Il loro unico compito è mappare questi fallimenti nei corretti codici di stato del protocollo di comunicazione (es. HTTP 400 Bad Request, 404 Not Found, 409 Conflict), restituendo al client messaggi chiari e sicuri.